

Tecdia SmartFix

AI-powered fault diagnostics for industrial machinery. Workers describe a symptom in plain English; SmartFix retrieves the relevant pages from indexed manuals, an LLM (Groq Llama-3.1-70B) explains what's happening, and a severity-weighted alert fires to managers when the issue is critical. Admin dashboard for machine management, shift logging, alerts, analytics, and audit.

Looking for the handover docs? They're in [docs/](#). Start with the README below or jump straight to any of them:

1. [API Documentation](#)
2. [Database Schema \(ER + tables\)](#)
3. [System Design](#)
4. [Routes & Endpoints](#)
5. [Tech & Dependencies](#)
6. [Codebase Ownership](#)

Quick start (dev)

```
# Backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # fill GROQ_API_KEY, RESEND_API_KEY, ADMIN_EMAILS
python3 -m uvicorn src.api:app --host 0.0.0.0 --port 8000

# Frontend (in another shell)
cd frontend
npm install
npm run dev # serves http://localhost:5173 with proxy to :8000
```

Open <http://localhost:5173>. To seed the admin dashboard with demo data: sign in, go to Analytics, click **Populate Demo Data**.

Project structure

```
smartfix/
├── src/                                # FastAPI backend
│   ├── api.py                        # All routes
│   ├── store.py                     # SQLite schema + helpers
│   ├── retriever.py / rag_pipeline.py / prompt_builder.py / llm_client.py
│   ├── audit.py / mailer.py / workstations.py / db.py
│   └── ingestion/parser_chunker.py  # PDF → chunks
├── frontend/                         # React + Vite SPA
│   ├── src/pages/                   # Landing, Admin, Chat, ...
│   ├── src/components/             # MachineCard, modals, banners
│   ├── src/context/                 # Auth, Machines, Alerts, Theme
│   ├── nginx.conf                  # Prod reverse proxy
│   └── Dockerfile                  # 2-stage Node build → nginx alpine
├── data/
│   ├── uploads/                    # archived machine PDFs (gitignored)
│   │   └── icons/                  # admin-uploaded custom icons
│   ├── workstations.json           # IP → machine bindings
│   └── audit.jsonl                 # append-only audit log
├── chroma_db/                      # vector index (gitignored)
└── smartfix.db                     # SQLite store (gitignored)
```

```
├── scripts/
│   ├── backup_sqlite.sh          # nightly SQLite backup
│   ├── build_index.py           # offline ingestion
│   └── demo.py                  # self-contained demo
├── postman/SmartFix.postman_collection.json
├── design/                      # design brief + factory hierarchy
├── docs/                        # ← handover docs
├── Dockerfile                   # backend image
├── docker-compose.yml          # backend + frontend + named volumes
├── requirements.txt
└── README.md
```

Production deploy

```
docker compose build
docker compose up -d
# frontend on host :80; backend internal-only on the docker network
```

Three named volumes (`chroma_db` , `data` , `sqlite_store`) persist state across container rebuilds. See [docs/03_SYSTEM_DESIGN.md](#) for the full architecture diagram.

Useful one-liners

```
# Inspect the SQLite store
sqlite3 smartfix.db ".tables"
sqlite3 smartfix.db "SELECT machine_id, phase, severity, created_at FROM shift_logs ORDER BY id DESC LIMIT 10;"

# Tail the audit log
tail -f data/audit.jsonl | python3 -m json.tool --no-ensure-ascii

# Manually back up SQLite
STORE_PATH=./smartfix.db BACKUP_DIR=./backups ./scripts/backup_sqlite.sh

# Re-build the vector index from local JSONL chunks (cache/*.jsonl)
python3 -m scripts.build_index
```

Status

- **Director's 3 features** (pre-shift checklist, end-of-shift parameter log, handoff banner): ✅ shipped end-to-end.
- **Admin power** (edit machines, void logs, ack/snooze alerts, runtime config, re-ingest, custom icons): ✅ shipped.
- **Analytics** (machine / category / severity / shift / time range / date range filters): ✅ shipped.
- **Production hardening** (alert persistence to SQLite, session-expiry handler, broader backup, tests): ⌚ P1 work, not started.

See [docs/06_CODEBASE_OWNERSHIP.md](#) for who to ping about which area.